# Compte Rendu : Projet Jeu de Bataille Navale

Jihene JAZIRI & Leo BOURGET

16 avril 2026

# Table des matières

**1 Introduction** **3**

**2 Fonctionnalités implémentées** **3**

**3 Choix d'architecture** **3**

**4 Difficultés rencontrées et solutions** **3**

**5 Répartition des tâches et retour d'expérience** **4**

5.1 Démarche et apprentissage . . . . . 4

5.2 Répartition du travail . . . . . 4

5.3 Retour d'expérience . . . . . 4

**6 Conclusion** **5**

# 1 Introduction

Ce rapport présente le développement de notre jeu de bateau réalisé en C++. L'objectif était de concevoir une application robuste en appliquant les principes de la programmation orientée objet.

Le concept du jeu consiste à piloter un navire afin de récupérer des objets dispersés dans l'eau. Le joueur est soumis à une contrainte de temps stricte : il doit collecter l'intégralité des objets avant la fin du chronomètre. Si le temps s'écoule alors qu'il reste encore des éléments en mer, la partie est perdue ; à l'inverse, si tous les objets sont récupérés dans le délai imparti, la victoire est remportée.

# 2 Fonctionnalités implémentées

Nous avons intégré les fonctionnalités suivantes au sein du jeu :
— Création et initialisation de la grille de jeu.
— Placement manuel et automatique des navires.
— Système de tir avec gestion des états (Touché, Coulé, À l'eau).
— Intelligence artificielle (basique/avancée) pour le mode solo.
— Sauvegarde et chargement de la partie.

# 3 Choix d'architecture

L'agencement du code suit une logique orientée objet pour assurer la modularité :

**Classe Bateau :** Classe parente gérant la taille et l'état des navires.

**Classe Grille :** Gère l'affichage et la matrice des cases.

**Classe Joueur :** Abstraction pour gérer l'humain et l'ordinateur.

**Gestionnaire de Jeu :** Boucle principale contrôlant le déroulement des tours.

# 4 Difficultés rencontrées et solutions

| **Difficulté** | **Solution mise en œuvre** |
|---|---|
| Gestion des collisions lors du placement des bateaux. | Implémentation d'une méthode de vérification récursive avant validation. |
| Fuites de mémoire avec les pointeurs. | Utilisation des `std::unique_ptr` et `std::shared_ptr` (Smart Pointers). |
| Affichage de la grille en console. | Utilisation de caractères ASCII et de codes couleurs pour la lisibilité. |

Table 1 – Tableau des problématiques techniques

# 5 Répartition des tâches et retour d'expérience

## 5.1 Démarche et apprentissage

Le projet a débuté par une phase d'appropriation des outils technologiques. Nous avons d'abord concentré nos efforts sur la compréhension des bibliothèques **SFML** (pour le rendu graphique) et **Libphysics** (pour la simulation physique).

Notre démarche a été progressive :

- **Première phase :** Création d'objets simples et mise en place de leurs premiers mouvements à l'écran.
- **Deuxième phase :** Intégration des forces physiques pour obtenir un comportement réaliste des entités, conformément au cahier des charges du projet.

## 5.2 Répartition du travail

Afin d'optimiser le développement, nous avons séparé les responsabilités selon deux axes principaux :

**Jihene JAZIRI :** S'est concentrée sur l'environnement **SFML**. Elle a conçu et implémenté toute la dimension visuelle du projet, notamment :
- La création et la gestion du menu principal.
- L'interface des paramètres.
- L'affichage global du jeu et l'intégration des assets graphiques.

**Léo BOURGET :** S'est spécialisé dans la partie **physique et logique de jeu**. Il a eu la charge de :
- La gestion dynamique du bateau et de ses mouvements.
- La génération des obstacles dans l'environnement marin.
- Le système de détection et de ramassage des objets par le navire.

## 5.3 Retour d'expérience

Ce projet a été une expérience très positive pour notre groupe. Nous avons particulièrement apprécié le concept du jeu, qui offrait un défi de programmation stimulant et concret.

Sur le plan humain, la collaboration au sein du binôme a été excellente. La communication fluide nous a permis de surmonter les obstacles techniques et de progresser efficacement. Cependant, le principal défi a été la gestion du temps : nous avions de nombreuses idées et options supplémentaires pour rendre le jeu plus complet et développé, mais le délai imparti nous a contraints à nous concentrer sur les fonctionnalités essentielles.

Le travail en groupe sur ce projet C++ nous a notamment permis de :

- **Développer une cohésion d'équipe :** Apprendre à coordonner nos efforts pour fusionner nos modules respectifs (physique et graphique).
- **Approfondir nos compétences techniques :** Maîtriser l'utilisation de bibliothèques externes comme SFML et Libphysics.
- **Gérer les priorités :** Faire des choix de conception pour livrer une version stable malgré les contraintes de temps.

# 6 Conclusion

Ce projet nous a permis de consolider nos bases en C++ tout en affrontant les contraintes réelles d'un développement collaboratif.